

Firma Genética: Detección de Secuencias Espejo en ADN

Análisis Algorítmico del "Longest Palindromic Substring"

Clase de Algoritmos y Estructuras de Datos

Contexto del Problema

Una **secuencia espejo** (palíndromo) es una cadena que se lee igual de izquierda a derecha que de derecha a izquierda.

En el ámbito de la bioinformática algorítmica, buscamos extraer la firma genética más larga de una muestra de ADN. El reto radica en la longitud extrema de la cadena:

- ✓ Alfabeto estricto: {A, C, G, T}
- ✓ Longitud máxima de cadena: **50,000**

caracteres.

Ejemplo visual: Encontrando el palíndromo "GCACG"

A T **G** **C** **A** **C** **G** T A



Definición Formal

Dado un string S compuesto únicamente por bases nitrogenadas, el objetivo es encontrar la subsecuencia contigua palindrómica S' de longitud máxima.

$$S \in \{A, C, G, T\}^*, \quad |S| \leq 50,000$$

$$\max_{0 \leq i \leq j < |S|} (j - i + 1) \quad \text{tal que} \quad S[i..j] = \text{reverse}(S[i..j])$$

Debido a los estrictos límites de tiempo de los jueces en línea (típicamente 1.0s), enfoques ingenuos superarán el límite de tiempo de ejecución (Time Limit Exceeded).

El Problema de la **Fuerza Bruta**

Antes de optimizar, evaluemos por qué las soluciones triviales fallan con $N = 50,000$:

- ▶ **Chequeo Exhaustivo $O(N^3)$** : Generar todas las subcadenas posibles y verificar si cada una es un palíndromo. Completamente inviable.
- ▶ **Expansión desde el Centro $O(N^2)$** : Iterar sobre cada carácter como centro y expandir hacia izquierda y derecha mientras coincidan.

Con $N = 50,000$, un algoritmo $O(N^2)$ requeriría aprox. **2.5 mil millones de operaciones**. En C++, esto toma ~2-5 segundos, resultando en TLE.

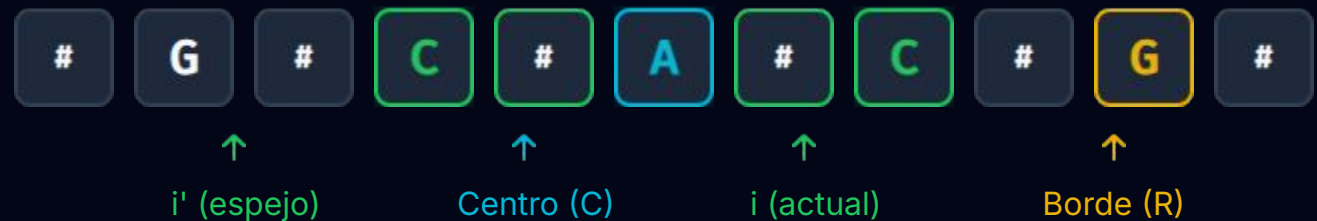


Técnica 1: Algoritmo de Manacher

Logra una complejidad temporal **estrictamente $O(N)$** aprovechando la simetría de los palíndromos encontrados.

- ▶ Transforma la cadena insertando '#' para tratar palíndromos pares e impares por igual.
- ▶ Mantiene registro del palíndromo que llega más a la **derecha (R)** y su **centro (C)**.
- ▶ Si la posición actual i está dentro de R, reutiliza el trabajo precalculado de su espejo i' .

Reutilización de simetría en S = "G C A C G"



Técnica 2: Rolling Hash + Búsqueda Binaria

1. Precomputación Hash $O(N)$

Calculamos arreglos de hash tanto para la cadena original (forward) como para su reversa (backward). Esto nos permite obtener el hash de cualquier subcadena en tiempo constante $O(1)$.

A C G

$\text{Hash}(S[0..2]) = \text{Hash_Rev}(S[N-3..N-1]) ?$

2. Búsqueda Binaria $O(N \log N)$

Iteramos sobre cada centro posible. Como la propiedad palindrómica es monotónica (si un palíndromo de longitud L existe, sus centros de $L-2$ también lo son), usamos Búsqueda Binaria sobre la longitud radial.

- ▶ Complejidad total: $O(N \log N)$
- ▶ **Riesgo:** Colisiones de Hash (requiere doble mod/base).

Técnica 3: Suffix Array + LCP Array

Las estructuras de sufijos ofrecen un enfoque puramente analítico del texto. Evaluamos la cadena concatenada de forma especial:

$$T = S + ' \# ' + \text{reverse}(S)$$

Construimos el **Suffix Array** y el **Longest Common Prefix (LCP) Array**. Para encontrar el palíndromo centrado en i , consultamos el LCP entre el sufijo que inicia en y_i su correspondiente sufijo invertido usando Range Minimum Query (RMQ).

$S = \text{"ACG"}, T = \text{"ACG\#GCA"}$

LCP	Sufijo
0	#GCA
0	A
1	ACG#GCA
0	CA
2	CG#GCA

Construcción $O(N \log N)$. Memoria pesada.

PSEUDOCÓDIGO — LA LÓGICA COMPLETA, PASO A PASO

ALGORITMO FirmaGenetica(S)

ENTRADA : S, cadena de ADN de longitud n

SALIDA : la subcadena palindrómica contigua más larga de S

—— PASO 1 · Construir un solo texto de trabajo ——

R ← S escrita al revés

T ← S + "#" + R + "\$"

Los símbolos # y \$ NO existen en {A,C,G,T} y valen menos que cualquier letra.

Sirven de muro: ninguna coincidencia podrá cruzar de S a R por accidente.

A partir de aquí ya no hay "dos cadenas": hay UNA sola, T.

—— PASO 2 · Ordenar todos los sufijos (el Suffix Array) ——

Tomar los |T| sufijos de T (un sufijo = "todo lo que queda desde la posición i")

Ordenarlos alfabéticamente, como palabras en un diccionario

SA ← la lista de las **posiciones donde empezaba** cada uno, en ese orden

Se guardan NÚMEROS, no texto: eso es lo que hace al método barato en memoria.

¿Por qué ordenar? Porque al ordenar, los sufijos que EMPIEZAN IGUAL quedan pegados uno junto a otro. Y "empezar igual" es exactamente lo que buscamos.

—— PASO 3 · Medir el parecido entre vecinos (arreglo LCP, algoritmo de Kasai) ——

PARA cada sufijo de la lista ordenada:

LCP[posición] ← cuántas letras comparte al inicio con el sufijo de ARRIBA

Truco de Kasai: si un sufijo comparte h letras con su vecino, el sufijo que empieza una posición más adelante comparte al menos h-1. Se arranca desde ahí en vez de recomenzar desde cero; por eso este paso entero cuesta solo O(N).

PSEUDOCÓDIGO — LA LÓGICA COMPLETA, PASO A PASO

—— PASO 4 · Preparar consultas instantáneas (Sparse Table) ——

Precalcular tablas de mínimos sobre LCP

DEFINIR $LCE(a, b) = \text{"¿cuántas letras comparten al inicio los sufijos } a \text{ y } b\text{"}$

= el MÍNIMO del arreglo LCP entre las posiciones que ocupan a y b en SA

Intuición: para ir de un sufijo a otro en la lista ordenada hay que pasar por todos los de en medio; el parecido se pierde en el eslabón más débil.

Con la Sparse Table ese mínimo se responde en tiempo CONSTANTE.

—— PASO 5 · Probar los $2n-1$ centros posibles ——

$mejor \leftarrow 1$; $inicio \leftarrow 0$ (una sola letra siempre es palíndromo)

PARA $i \leftarrow 0$ HASTA $n-1$: — centros IMPARES: el centro es la letra i

$k \leftarrow LCE(\text{sufijo de } S \text{ en } i, \text{sufijo de } R \text{ en } n-1-i)$

$k = \text{cuántos pasos se puede avanzar hacia los dos lados sin romper la simetría}$

 SI $2k-1 > mejor$ ENTONCES $mejor \leftarrow 2k-1$; $inicio \leftarrow i-k+1$

PARA $i \leftarrow 1$ HASTA $n-1$: — centros PARES: el centro cae ENTRE $i-1$ e i

$k \leftarrow LCE(\text{sufijo de } S \text{ en } i, \text{sufijo de } R \text{ en } n-i)$

 SI $2k > mejor$ ENTONCES $mejor \leftarrow 2k$; $inicio \leftarrow i-k$

DEVOLVER $S[inicio .. inicio + mejor - 1]$

FIN

Técnica 4: Programación Dinámica (LCS)

El problema puede reducirse a encontrar el **Longest**

Common Substring entre S (reversa).

Se construye una matriz DP bidimensional. Si , entonces .

$$S[i] == S'[j]$$

$$DP[i][j] = DP[i-1][j-1] + 1$$

Riesgo Crítico: Requiere verificar que los índices coincidan espacialmente. Ejemplo: en "ABACDFGDCABA", LCS encuentra "ABACD" y "DCABA" como iguales, pero no forman un palíndromo real en S.

DP Matriz ($S="GCACG"$, $S'="GCACG"$)

	G	C	A	C	G
G	1	0	0	0	1
C	0	2	0	1	0
A	0	0	3	0	0
C	0	1	0	4	0
G	1	0	0	0	5

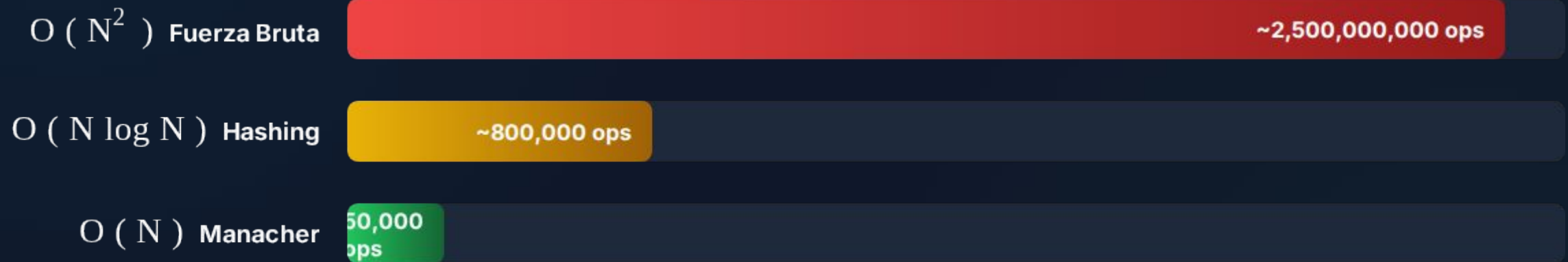
Complejidad Espacial $O(N^2)$: ¡Inviabile para $N=50k$!

Evaluación Algorítmica

Técnica	 Tiempo	 Espacio	 Dificultad	Riesgos / Desventajas
Fuerza Bruta Expandida	$O(N^2)$	$O(1)$	Baja	Time Limit Exceeded (TLE)
DP (LCS)	$O(N^2)$	$O(N^2)$ (o $O(N)$)	Media	Memory Limit Exceeded (MLE) / Falsos positivos
Hashing + Binary Search	$O(N \log N)$	$O(N)$	Alta	Colisiones de Hash, constante oculta alta
Suffix Array + LCP	$O(N \log N)$	$O(N)$ (pesado)	Muy Alta	Implementación larga y propensa a bugs
Algoritmo de Manacher	$O(N)$	$O(N)$	Media	Manejo cuidadoso de índices (inserción de '#')

Rendimiento en $N = 50,000$ (Juez en Línea)

Comparación de operaciones base requeridas procesando una cadena de ADN de longitud máxima.



Benchmark del Equipo: La solución implementada con Manacher procesó el test máximo ($N=50,000$) en tan solo **7 milisegundos**, superando el límite de 1.0s holgadamente.

Decisión Final: **Manacher**

Para la resolución del problema competitivo, el equipo optó por el Algoritmo de Manacher por las siguientes razones definitivas:

- ▶ **Complejidad Óptima:** Garantiza tiempo lineal $O(N)$ estricto, sin constantes altas como en Suffix Arrays.
- ▶ **Cero Colisiones:** A diferencia de Rolling Hash, es un algoritmo determinista; no existe riesgo de fallar casos trampa por colisiones (falsos positivos).
- ▶ **Eficiencia Espacial:** Solo requiere un arreglo de enteros del tamaño $2N + 1$, evitando el overhead de memoria catastrófico de la Programación Dinámica bidimensional.



Demo del Código (C++)

manacher.cpp

```
string longestPalindrome(string s) { // 1. Transformación (A → #A#) string T = "^#"; for (char c : s) { T += c; T += "#";  
} T += "$"; int n = T.length(); vector<int> P(n, 0); int C = 0, R = 0; // 2. Expansión con reutilización de simetría for  
(int i = 1; i < n - 1; i++) { int i_mirror = 2 * C - i; if (R > i) P[i] = min(R - i, P[i_mirror]); while (T[i + 1 + P[i]]  
= T[i - 1 - P[i]]) P[i]++; if (i + P[i] > R) { C = i; R = i + P[i]; } } // 3. Extracción del máximo (omitted for brevity)  
return result; }
```

Conclusiones y Aprendizajes



Contexto Biológico

Resolver problemas abstractos de cadenas tiene aplicaciones directas y críticas en bioinformática, como la búsqueda de sitios de restricción o firmas genéticas complejas.



El Límite del Escalamiento

Comprendimos de forma empírica por qué un algoritmo $O(N^2)$ es inútil para Big Data ($N=50k+$) y la necesidad absoluta de encontrar optimizaciones lineales.



Elegancia vs Complejidad

Aunque Suffix Arrays resuelven múltiples problemas, Manacher demostró que un algoritmo especializado y más simple suele ser la herramienta más elegante y rápida para una tarea específica.

Image Sources



https://elements-resized.envatousercontent.com/elements-video-cover-images/2733b6a8-609d-4389-bb05-0b79045fdbdc/video_preview/video_preview_0000.jpg?w=500&cf_fit=cover&q=85&format=auto&s=83afc919d8648df7bcd28062f2c4acd3b148473d2d4d5dbb920daf750f151f47

Source: elements.envato.com



https://images.stockcake.com/public/9/e/3/9e383b46-cf51-4613-a042-2f6d0b0041a4_medium/ai-network-visualization-stockcake.jpg

Source: stockcake.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/3d9adca2-b53e-440b-945b-bba1f552bbb7/video_preview/video_preview_0000.jpg?w=500&cf_fit=cover&q=85&format=auto&s=f5ddce7ba7bce3b43a4f822bc79ce902b8cb02ef6749bad21a558328386a280a

Source: elements.envato.com



https://images.stockcake.com/public/7/e/c/7ecce22e-c7d2-4cf0-8764-5eaaf8b09884_medium/luminous-dna-spiral-stockcake.jpg

Source: stockcake.com